```
---
title: "Presentación Módulo 2"
output: ioslides_presentation
date: "`r Sys.Date()`"
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = FALSE)
```
¿Qué es el Método Burbuja (Bubble Sort)?
Es, estadísticamente hablando, un algoritmo bastante ineficiente para ordenar una lista de números. Permite tomar un valor, compararlo con el siguiente y moverlo, hasta lograr que toda la lista esté ordenada.

* Tenemos una lista de números caótica.
* Comparamos el primero con el segundo. Si el primero es mayor que el segundo, los cambiamos de lugar.
* Avanzamos hasta llegar al último número.

Se llama Método Burbuja porque los números más grandes y pesados "flotan" hacia el final de la lista como si fueran burbujas. Lento y doloroso.

Función nativa de R
Por suerte, R es un lenguaje medianamente inteligente y ya trae una función que hace exactamente lo mismo, pero sin derretir el procesador.

La función `sort()` es la que permite que ordenemos los números eficientemente bajo el capó.

Pero, ¿según quién la función `sort()` es mejor que el Método Burbuja? No es una opinión, lo dicen los Benchmarks.

Pero, ¿qué chirolas es un Benchmark?
Piensen en un benchmark como un cronómetro glorificado. Es básicamente una función que nos va a permitir evaluar cuánto tiempo de su vida pierde la computadora en correr nuestro programa.

En nuestro caso, vamos a usar la biblioteca **microbenchmark** (aunque **rbenchmark** cumple la misma función). Esta biblioteca nos permite poner a competir múltiples fragmentos de código R y comparar sus tiempos de ejecución en milisegundos.

Complejidad temporal del algoritmo burbuja
Para demostrar esto, vamos a generar una lista de apenas 100 números aleatorios y ordenarlos con cada método.

El algoritmo burbuja tiene una complejidad temporal cuadrática, conocida como $\mathcal{O}(n^2)$. En términos simples: para apenas 100 elementos, el algoritmo tiene la audacia de hacer casi 10.000 operaciones en su peor escenario.

En resumen: Nuestro bloque de código manual se va a demorar una eternidad relativa, mientras que `sort()` lo va a resolver antes de que ustedes parpadeen.

El Código del programa

```r
# Instalar paquete
if (!require(microbenchmark)) {
 install.packages("microbenchmark")
 library(microbenchmark)
}
# Generar muestra
set.seed(123)
n <- 100
vector <- sample(1:100000, n, replace = TRUE)
```
El Código del programa

```r
#Implementación de ordenamiento burbuja
bubble_sort <- function(x) {
 n <- length(x)
```
```

```
 for (i in 1:(n-1)) {
 for (j in 1:(n-i)) {
 if (x[j] > x[j+1]) {
 temp <- x[j]
 x[j] <- x[j+1]
 x[j+1] <- temp
 }
 }
 }
 return(x)
}
```

El Código del programa

```r
#Benchmark (OJO: burbuja es muy lento)
resultado <- microbenchmark(
 burbuja = bubble_sort(vector),
 sort_R = sort(vector),
 times = 5
)
print(resultado)
```

Resultado del programa

```{r}
# Instalar paquete
if (!require(microbenchmark)) {
 install.packages("microbenchmark")
 library(microbenchmark)
}
# Generar muestra
set.seed(123)
n <- 100
vector <- sample(1:100000, n, replace = TRUE)
#Implementación de ordenamiento burbuja
bubble_sort <- function(x) {
 n <- length(x)
 for (i in 1:(n-1)) {
 for (j in 1:(n-i)) {
 if (x[j] > x[j+1]) {
 temp <- x[j]
 x[j] <- x[j+1]
 x[j+1] <- temp
 }
 }
 }
 return(x)
}
#Benchmark (OJO: burbuja es muy lento)
resultado <- microbenchmark(
 burbuja = bubble_sort(vector),
 sort_R = sort(vector),
 times = 5
)
print(resultado)
```

Desglose - Parte 1: Nuestro vector
* `set.seed(123)`: Fija la semilla aleatoria. Genera un vector al azar, pero asegura que siempre sea el mismo. Así este experimento no nos explota en la cara y se puede replicar.

* `n <- 100`: La cantidad de víctimas (elementos) que tendrá el vector.

* `vector <- sample(1:100000, n, replace = TRUE)`: Sacamos nuestra muestra de 100 elementos, usando números del 1 al 100.000, y permitimos que se repitan.

Desglose - Parte 2: La tragedia del Algoritmo Burbuja
Bautizamos esta función con `bubble_sort <- function(x)`. La `x` entre los paréntesis es el vector desordenado que va a sufrir el proceso.

* `n <- length(x)`: Contamos los elementos. Lo necesitamos para saber cuántas veces vamos a torturar a la máquina repitiendo el proceso.
* `for (i in 1:(n-1))`: Nuestro **Bucle Externo**. Le decimos a la computadora cuántas pasadas completas tiene que darle a la lista. Si tenemos 10 números y ya ordenamos 9, el último ya está acomodado, por eso es `n-1`.

Desglose - Parte 2: La tragedia del Algoritmo Burbuja

* `for (j in 1:(n-i))`: El **Bucle Interno**. La `j` es la posición actual. Para ahorrar una miseria de tiempo, se va acortando (`n-i`). En cada pasada, el número más grande ya quedó al final, así que reducimos los datos que tenemos que volver a revisar.

(El núcleo del intercambio)
```r
 if (x[j] > x[j+1]) {
 temp <- x[j]
 x[j] <- x[j+1]
 x[j+1] <- temp
 }
```

Desglose - Parte 2: La tragedia del Algoritmo Burbuja

Comparamos el número actual con su vecino. Si el de la izquierda es mayor, están mal. Los intercambiamos usando una variable temporal `temp` como silla de espera para no borrar los datos.

* `return(x)`: Toma el vector ordenado y nos lo devuelve. Aplausos lentos.

Parte 3: El Benchmark (La Carrera)

Aquí llamamos al juez implacable:
```r
resultado <- microbenchmark(
```
* `burbuja = bubble_sort(vector)`: Le decimos al cronómetro "che, corré mi código lento y llamalo burbuja".
* `sort_R = sort(vector)`: Le decimos "ahora corré la función profesional de R".
* `times = 5`: Repetimos esta miseria 5 veces para calcular un promedio decente y que no sea una casualidad.

Por último, `print(resultado)` imprime la tabla que demuestra matemáticamente sort() es mas rapido que bubble_sort.